An Adaptive Multi-Agent System with Data-Driven Heuristic Orchestration for Solving 1D/2D Bin-Packing Problems

Sam Guessoum

A Thesis
Submitted to the Faculty
of the
WORCESTER POLYTECHNIC INSTITUTE
in partial fulfillment of the requirements for the
Degree of Master of Science
in
Computer Science

May 2026

APPROVED:

Professor Chun-Kit Ngan
Advisor
Worcester Polytechnic Institute

Dr. Rolf Bardeli
Committee Member
thyssenkrupp Materials Services GmbH

Abstract

This paper introduces an advanced Multi-Agent System (MAS) featuring a five-agent architecture designed to tackle bin packing problems (BPPs). This framework seamlessly integrates machine learning-driven heuristic selection, adaptive search termination, and unified dimensionality handling. Our end-to-end MAS architecture closes the performance gap between heuristics, the tuning overhead of metaheuristics, and the computational burden of exact methods by coordinating five specialized agents. Each agent is classified using established AI agent taxonomy. We rigorously evaluate our framework against five heuristics, three metaheuristics, and one exact method (Branch and Bound (B&B)) across over 3,000 instances from 12 diverse datasets. The MAS achieves a 9% average improvement in bin utilization over all heuristics and metaheuristics. It also proves 10× faster than B&B, consistently maintaining an optimality gap of ≤2% and reducing bin usage. Our statistical validation confirms significant outperformance against heuristic ($\alpha = 0.05$, p-values 0.0001-0.0011) and metaheuristic baselines (p-values 0.0038-0.0061) in optimality gap and bin usage. Crucially, there is no significant difference from B&B (p=0.0674) despite being 10× faster. We also develop a web-based Proof-of-Concept prototype, providing an interactive environment for solving both 1D and 2D BPPs.

Acknowledgments

The authors would like to acknowledge thyssenkrupp Materials Services GmbH for providing access to industrial bin packing datasets that supported the experimental evaluation conducted in this research.

The authors also acknowledge the computational resources provided by the Turing Research Cluster at Worcester Polytechnic Institute, which enabled large-scale experimentation.

In addition, the authors thank Gurobi Optimization for providing an academic license used in the optimization components of this work.

Finally, the authors express gratitude to all individuals and institutions that contributed directly or indirectly to the completion of this research.

Contents

List of Tables

List of Figures

Chapter 1

Introduction

The Bin Packing Problem (BPP) is a classical NP-hard combinatorial optimization problem that has been extensively studied due to its theoretical complexity and wide range of practical applications [7, 6, 19]. The objective of BPP is to pack a set of items into the minimum number of bins without exceeding their capacity constraints. This problem arises in various domains, including logistics, manufacturing, supply chain management, and resource allocation, where efficient utilization of limited capacity is critical.

In modern industrial environments, inefficient packing leads to significant economic and operational costs. For example, inefficiencies in transportation and logistics have been shown to result in substantial financial losses due to wasted space and suboptimal resource utilization. Industry solutions such as DHL's OptiCarton have demonstrated that improved packing strategies can reduce empty space by up to 65%, leading to significant reductions in ship-

ping costs [11]. Similarly, large-scale manufacturing systems, such as those operated by companies like Caterpillar, face challenges related to inefficient material handling and packing, which directly impact production efficiency and operational reliability.

Over the years, three major classes of approaches have been developed to address the bin packing problem: heuristic methods, exact algorithms, and metaheuristic techniques. Classical constructive heuristics, such as First Fit (FF) and Best Fit (BF), assign items sequentially based on greedy placement rules [7]. Variants such as First Fit Decreasing (FFD) and Best Fit Decreasing (BFD) improve performance by sorting items prior to assignment, leading to better packing efficiency in practice [30]. Despite their computational efficiency, these methods suffer from myopic decision-making and often fail to capture global packing structures.

Exact methods, such as Branch and Bound (B&B), guarantee optimal solutions by systematically exploring and pruning the search space [25]. However, these approaches exhibit exponential time complexity and are therefore impractical for large-scale or real-time applications. Metaheuristic algorithms, including Simulated Annealing (SA) [21], Genetic Algorithms, and Cuckoo Search (CS) [40], attempt to balance solution quality and computational efficiency by employing stochastic search strategies. While these methods can produce near-optimal solutions, they typically require extensive parameter tuning and may exhibit inconsistent runtime performance. More broadly, metaheuristic frameworks have been extensively studied for

combinatorial optimization problems, offering flexible and scalable solution strategies [34]. However, their effectiveness often depends on careful parameter tuning and problem-specific adaptation.

Despite significant progress, a fundamental challenge remains: no single method simultaneously achieves high solution quality, low computational cost, and adaptability across diverse problem instances. Heuristic methods are fast but suboptimal, exact methods are optimal but computationally expensive, and metaheuristics provide a compromise but lack robustness and consistency. This trade-off is particularly problematic in real-time industrial applications, where both solution quality and computational efficiency are critical.

To address this challenge, this thesis proposes an Adaptive Multi-Agent System (MAS) for solving one-dimensional (1D) and two-dimensional (2D) bin packing problems. The proposed framework integrates heuristic, metaheuristic, and exact optimization techniques within a unified decision-driven architecture composed of five specialized agents. Each agent is responsible for a distinct phase of the optimization process, enabling coordinated and adaptive problem solving.

A key component of the proposed system is a Reinforcement Learning (RL)-based heuristic selector [33, 38, 1], which dynamically selects the most appropriate constructive heuristic based on instance-level features. The selected heuristic generates an initial feasible solution, which is subsequently refined by a metaheuristic optimization agent using Cuckoo Search. If fur-

ther improvement is required, an exact method agent applies a time-bounded Branch and Bound procedure to enhance solution quality. A central pipeline orchestrator coordinates the interaction among agents, manages computational budgets, and enables adaptive learning through feedback mechanisms.

The proposed framework is designed to achieve three primary objectives: (1) near-optimal solution quality, (2) sub-second computational performance suitable for real-time applications, and (3) adaptability to heterogeneous problem instances. Unlike traditional monolithic optimization approaches, the multi-agent architecture enables modularity, extensibility, and intelligent coordination among heterogeneous solution strategies.

The effectiveness of the proposed system is evaluated on more than 3,000 instances across twelve benchmark datasets, including both 1D and 2D bin packing problems [10]. Experimental results demonstrate that the proposed MAS consistently outperforms classical heuristics and metaheuristics, achieving up to 9% improvement in bin utilization while maintaining an optimality gap of less than 2%. Furthermore, the system achieves up to 10× faster runtime compared to exact methods such as Branch and Bound, making it suitable for real-time deployment scenarios.

In addition to algorithmic contributions, this thesis presents a web-based proof-of-concept prototype that enables users to configure problem instances, execute optimization workflows, and visualize packing solutions in real time. This prototype demonstrates the practical applicability of the proposed framework in industrial contexts such as logistics, manufacturing, and e-commerce

fulfillment.

1.1 Research Questions

This thesis is guided by the following research questions:

- How can optimization strategies for bin packing problems be dynamically selected based on instance characteristics?
- Can a hybrid framework combining heuristics, metaheuristics, and exact methods achieve both high solution quality and low runtime?
- How can reinforcement learning be effectively integrated into combinatorial optimization pipelines for real-time decision-making?
- What are the advantages of a multi-agent architecture in coordinating heterogeneous optimization techniques?

1.2 Contributions

The main contributions of this thesis are summarized as follows:

- Development of an adaptive Multi-Agent System (MAS) for solving 1D and 2D bin packing problems.
- Comprehensive experimental evaluation on large-scale benchmark datasets with statistical validation.

- Implementation of a web-based prototype for real-time visualization and industrial application.

1.3 Thesis Organization

The remainder of this thesis is organized as follows. Chapter 2 presents the theoretical background of bin packing problems, including heuristic, metaheuristic, and exact approaches, as well as an overview of reinforcement learning and multi-agent systems. Chapter 3 reviews existing literature related to bin packing and hybrid optimization frameworks. Chapter 4 introduces the proposed adaptive Multi-Agent System and details its architecture and components. Chapter 5 presents the mathematical formulation and interaction mechanisms among agents. Chapter 6 provides an extensive experimental evaluation and analysis of the proposed framework. Chapter 7 describes the system prototype and its industrial applications. Chapter 8 concludes the thesis and outlines directions for future research.

Chapter 2

Background

This chapter provides the theoretical and methodological foundations necessary to understand the bin packing problem and the proposed solution framework. It covers the formulation of bin packing problems, classical solution approaches, metaheuristic optimization techniques, reinforcement learning fundamentals, and multi-agent systems.

2.1 Bin Packing Problem

The Bin Packing Problem (BPP) is a well-known NP-hard combinatorial optimization problem that aims to pack a set of items into the minimum number of bins without exceeding capacity constraints. Formally, given a set of items $I = \{i_1, i_2, ..., i_n\}$ with associated sizes and a set of bins B with flexible capacities, the objective is to minimize the number of bins used while

ensuring that the total size of items assigned to each bin does not exceed its capacity.

BPP can be categorized into different variants depending on dimensionality and constraints. Bin packing problems can also be classified within the broader family of cutting and packing problems based on dimensionality, assignment structure, and geometric constraints [36]. In the one-dimensional (1D) BPP, each item is characterized by a single size (e.g., length), while in the two-dimensional (2D) BPP, items have both width and height, introducing geometric placement constraints such as non-overlapping and boundary conditions. The 2D variant is significantly more complex due to the combinatorial explosion of possible placements and orientations.

The BPP is NP-hard in the strong sense, meaning that no polynomial-time algorithm is known to solve all instances optimally [7]. As a result, various approximation and heuristic approaches have been developed to address practical instances efficiently.

2.2 Heuristic Methods

Heuristic methods are among the earliest and most widely used approaches for solving BPP due to their simplicity and computational efficiency. These methods construct solutions incrementally based on greedy decision rules.

2.2.1 First Fit and Best Fit

The First Fit (FF) algorithm assigns each item to the first bin that can accommodate it, while the Best Fit (BF) algorithm places the item into the bin that will have the least remaining capacity after placement [7, 6]. Both methods operate with a worst-case time complexity of $O(n^2)$, as each item may require scanning all existing bins.

2.2.2 Decreasing Variants

To improve packing efficiency, decreasing variants such as First Fit Decreasing (FFD) and Best Fit Decreasing (BFD) sort items in descending order before applying the heuristic [30]. These approaches significantly reduce the number of bins used in practice and provide improved approximation guarantees.

2.2.3 Limitations of Heuristics

Despite their efficiency, heuristic methods suffer from several limitations. They rely on local decision-making and do not consider global packing structures, which often leads to suboptimal solutions. Additionally, they are not adaptable to varying instance characteristics and cannot easily incorporate complex constraints.

2.3 Exact Methods

Exact methods aim to find optimal solutions by exhaustively exploring the solution space. One of the most prominent exact approaches for BPP is the Branch and Bound (B&B) algorithm.

2.3.1 Branch and Bound

Branch and Bound systematically divides the problem into subproblems (branching) and computes bounds to eliminate suboptimal solutions (bounding). By pruning branches that cannot lead to better solutions, B&B reduces the search space [25].

However, the worst-case time complexity of B&B is exponential, typically $O(2^n)$, making it unsuitable for large-scale instances or real-time applications. While B&B guarantees optimality, its computational cost limits its practical applicability.

2.4 Metaheuristic Methods

Metaheuristic algorithms provide a balance between solution quality and computational efficiency by exploring the search space using stochastic and adaptive strategies.

2.4.1 Simulated Annealing

Simulated Annealing (SA) is inspired by the annealing process in metallurgy and uses probabilistic acceptance of worse solutions to escape local optima [21, 34]. It is widely used for combinatorial optimization problems due to its simplicity and effectiveness.

2.4.2 Genetic Algorithms

Genetic Algorithms (GA) simulate the process of natural selection by evolving a population of candidate solutions through operations such as selection, crossover, and mutation. They are particularly effective for exploring large search spaces but require careful parameter tuning.

2.4.3 Cuckoo Search

Cuckoo Search (CS) is a population-based metaheuristic inspired by the brood parasitism behavior of cuckoos [40]. It employs Lévy flights to perform global search and has been shown to be effective for optimization problems due to its ability to balance exploration and exploitation.

2.4.4 Limitations of Metaheuristics

Although metaheuristics can produce near-optimal solutions, they often require extensive parameter tuning and may exhibit unpredictable runtime behavior. Additionally, they do not provide optimality guarantees.

2.5 Reinforcement Learning

Reinforcement Learning (RL) is a machine learning paradigm in which an agent learns to make decisions by interacting with an environment and receiving feedback in the form of rewards.

2.5.1 Fundamentals of Reinforcement Learning

In RL, an agent observes a state s, selects an action a, and receives a reward r. The objective is to learn a policy that maximizes the cumulative reward over time [33, 38, 27, 1]. The learning process is typically modeled as a Markov Decision Process (MDP).

2.5.2 RL for Combinatorial Optimization

Recent advances have demonstrated the potential of RL for solving combinatorial optimization problems by learning decision policies from data. In the context of BPP, RL can be used to select heuristics or guide search strategies based on instance characteristics.

In this thesis, RL is used as a lightweight decision mechanism to select the most appropriate heuristic based on instance-level features, enabling adaptive and data-driven optimization.

2.6 Multi-Agent Systems

A Multi-Agent System (MAS) consists of multiple interacting agents, each capable of making decisions and performing tasks independently.

2.6.1 Agent Types

Agents can be categorized into different types, including:

- Reflex agents, which act based on predefined rules.
- Goal-based agents, which aim to achieve specific objectives.
- Utility-based agents, which optimize a performance measure.
- Learning agents, which improve their behavior over time.

2.6.2 Coordination and Cooperation

In MAS, agents coordinate and cooperate to solve complex problems that are difficult for a single agent to handle. This enables modularity, scalability, and flexibility in system design.

2.6.3 MAS in Optimization

Multi-Agent Systems (MAS) have been widely studied as a paradigm for distributed and intelligent problem solving [39, 18]. Multi-agent approaches

have been applied to optimization problems to enable distributed decision-making and hybrid solution strategies. By decomposing the problem into smaller tasks handled by specialized agents, MAS frameworks can effectively integrate multiple optimization paradigms.

2.7 Summary

This chapter reviewed the fundamental concepts underlying bin packing problems and their solution approaches, including heuristic, exact, and meta-heuristic methods. It also introduced reinforcement learning and multi-agent systems as key components of the proposed framework. These concepts provide the foundation for the adaptive multi-agent system presented in the following chapters.

Chapter 3

Related Work

This chapter reviews existing approaches for solving bin packing problems and situates the proposed work within the broader research landscape. The discussion is organized into five categories: classical heuristic methods, exact algorithms, metaheuristic techniques, learning-based approaches, and multi-agent systems.

3.1 Classical Heuristic Approaches

Heuristic algorithms have been widely used for solving bin packing problems due to their simplicity and computational efficiency. Early work by Coffman et al. [7, 6] introduced classical heuristics such as First Fit (FF) and Best Fit (BF), which assign items to bins based on greedy placement rules. These methods operate efficiently but often produce suboptimal solutions due to

their local decision-making nature.

Subsequent improvements introduced decreasing variants such as First Fit Decreasing (FFD) and Best Fit Decreasing (BFD), which sort items in descending order prior to assignment [30]. These methods significantly improve solution quality in practice and provide better approximation guarantees. Additional heuristics, such as Better Fit (BeF), aim to further reduce unused space by selecting bins that best match item dimensions.

Despite their widespread use, heuristic approaches remain fundamentally limited by their inability to consider global packing structures and adapt to instance-specific characteristics. As a result, their performance degrades in complex or heterogeneous problem settings.

3.2 Exact Methods

Exact algorithms aim to compute optimal solutions by exhaustively exploring the solution space. One of the most widely studied exact approaches for bin packing is the Branch and Bound (B&B) algorithm [25]. This method systematically explores possible assignments while pruning suboptimal branches using bounding techniques.

Although exact methods guarantee optimality, they suffer from exponential time complexity, making them impractical for large-scale problems. As problem size increases, the computational cost becomes prohibitive, limiting their applicability in real-time or industrial scenarios.

Integer Linear Programming (ILP) formulations have also been used to model bin packing problems. These formulations provide a flexible framework for incorporating constraints but often require significant computational resources and are typically solved using commercial solvers.

3.3 Metaheuristic Approaches

Metaheuristic algorithms have been extensively applied to bin packing problems to overcome the limitations of both heuristics and exact methods. These approaches use stochastic search strategies to explore the solution space and escape local optima.

Simulated Annealing (SA) is one of the earliest metaheuristic methods applied to combinatorial optimization. It probabilistically accepts worse solutions to avoid premature convergence. Genetic Algorithms (GA) use evolutionary principles to iteratively improve a population of solutions through selection, crossover, and mutation.

More recent approaches include Improved Grouping Genetic Algorithms (IGGA) and Cuckoo Search (CS), which have demonstrated strong performance in bin packing applications. Cuckoo Search, introduced by Yang and Deb [40], employs Lévy flight-based exploration to balance global and local search effectively.

Despite their advantages, metaheuristic methods often require careful parameter tuning and may exhibit unstable performance across different prob-

lem instances. Furthermore, they do not provide guarantees of optimality. More comprehensive studies of metaheuristic design and performance can be found in [34].

3.4 Hyper-Heuristics and Algorithm Selection

To address the limitations of individual heuristics, hyper-heuristic approaches have been proposed to select or generate heuristics dynamically. These methods aim to automate the process of choosing the most suitable heuristic based on problem characteristics.

Burke et al. [5] provide a comprehensive survey of hyper-heuristics, highlighting their ability to generalize across different problem domains. The algorithm selection problem, introduced by Rice [29], formalizes the idea of selecting the best algorithm for a given problem instance based on its features.

In the context of bin packing, hyper-heuristic methods have been used to adaptively select heuristics during the solution process. However, many existing approaches rely on predefined rules or offline training and lack the ability to integrate multiple optimization paradigms within a unified framework.

3.5 Learning-Based Approaches

Recent advances in machine learning have led to the development of learning-based methods for combinatorial optimization. Reinforcement Learning (RL), in particular, has shown promise in learning decision policies for complex problems.

Deep learning approaches, such as Deep Q-Networks (DQN) [27, 1, 20, 15], have demonstrated the ability to learn effective policies from data. In bin packing, learning-based methods have been used to predict packing strategies or guide search processes.

However, many of these approaches require large amounts of training data and computational resources, making them difficult to deploy in real-time applications. Additionally, they often focus on replacing traditional algorithms rather than complementing them.

3.6 Multi-Agent Systems in Optimization

Multi-Agent Systems (MAS) have been explored as a framework for solving complex optimization problems by decomposing them into smaller tasks handled by specialized agents. Early work by Horký [17] introduced a multi-agent approach for bin packing, combining heuristic strategies with integer programming.

While these approaches demonstrate the potential of MAS in optimization, they often rely on static heuristics and lack adaptability to changing

problem conditions. Furthermore, many existing MAS frameworks do not incorporate learning mechanisms or dynamic coordination among agents.

3.7 Two-Dimensional Bin Packing

In the context of two-dimensional bin packing, additional geometric constraints such as item placement, rotation, and non-overlapping conditions significantly increase problem complexity. Exact and hybrid approaches have been developed to address these challenges, including advanced placement strategies and mathematical formulations [22, 4, 2].

These methods extend classical bin packing approaches by incorporating spatial reasoning and geometric feasibility, making them particularly relevant for industrial applications such as cutting stock and sheet metal processing.

3.8 Research Gap

Despite extensive research, existing approaches exhibit several limitations. Heuristic methods are fast but lack solution quality, exact methods guarantee optimality but are computationally expensive, and metaheuristics provide a compromise but require extensive tuning and lack consistency. Hyper-heuristic and learning-based approaches improve adaptability but often operate in isolation and do not integrate multiple optimization paradigms.

Moreover, existing multi-agent approaches for bin packing are typically

static and do not leverage data-driven decision-making or adaptive coordination mechanisms. There is a lack of unified frameworks that combine heuristic selection, metaheuristic refinement, and exact optimization within a single adaptive system.

3.9 Summary

This chapter reviewed the major approaches for solving bin packing problems, including heuristic, exact, metaheuristic, learning-based, and multi-agent methods. While each approach has its strengths, none fully addresses the trade-off between solution quality, computational efficiency, and adaptability. These limitations motivate the development of the adaptive multi-agent system proposed in this thesis, which integrates multiple optimization strategies within a unified and data-driven framework.

Chapter 4

Adaptive Multi-Agent System Framework

Multi-agent architectures have been widely applied to complex optimization problems due to their modularity and scalability [39]. This chapter presents the proposed Adaptive Multi-Agent System (MAS) for solving one-dimensional (1D) and two-dimensional (2D) bin packing problems. The framework in Figure 4.1 integrates heuristic, metaheuristic, and exact optimization techniques within a unified architecture composed of specialized agents. The objective is to balance solution quality, computational efficiency, and adaptability across diverse problem instances.

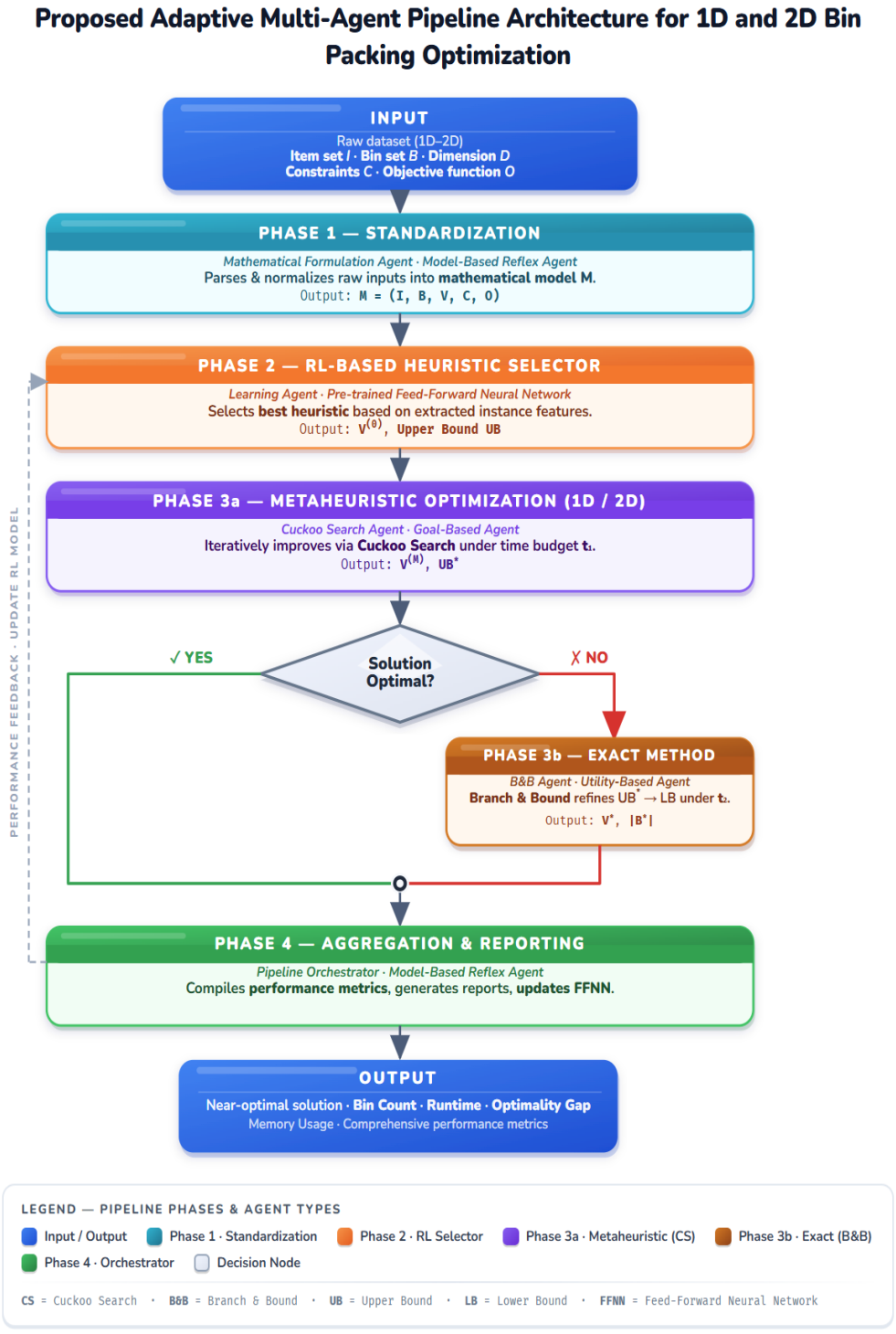

Figure 4.1: Multi-Agent Pipeline Architecture for 1D and 2D Bin Packing Problems.

4.1 Overview of the Framework

The proposed MAS is designed as a sequential, decision-driven pipeline consisting of five specialized agents. Each agent is responsible for a distinct stage of the optimization process, enabling modular and coordinated problem solving.

The framework operates on a unified mathematical representation of the bin packing problem, ensuring consistency across all stages of the optimization process. The pipeline begins with problem standardization, followed by heuristic selection, metaheuristic refinement, and optional exact optimization. A central orchestrator manages the interaction between agents and controls computational budgets.

This design enables the system to dynamically adapt its behavior based on problem characteristics and runtime constraints, providing a flexible and scalable solution for both 1D and 2D bin packing problems.

4.2 Input

To ensure consistency across all agents, the bin packing problem is represented using a unified mathematical model:

$$M = \langle I, B, V, C, O \rangle$$

where:

- I denotes the set of items,
- B denotes the set of bins,
- V represents the decision variables,
- C represents the set of constraints,
- O is the objective function minimizing bin usage.

This unified representation allows all agents to operate on the same problem structure, enabling seamless interaction and information sharing throughout the pipeline.

4.3 Phase 1: Model Construction

The pipeline begins with the *Mathematical Formulation Agent*. This model-based reflex agent standardizes the input instance by constructing the unified optimization model M. Given the item set I, bin set B, dimensionality D, constraint set C, and objective function O, the agent instantiates the appropriate decision variables V satisfying all constraints in C. For $D = 1D$, width parameters are normalized and only linear capacity constraints are enforced. For $D = 2D$, placement variables, rotation variables, boundary constraints, and non-overlapping constraints are instantiated. The output of this phase is a valid optimization model that is ready for solution generation.

4.4 Phase 2: Heuristic Selection

The constructed model M is passed to the *RL-Based Heuristic Selector*, functioning as a learning agent. This agent extracts instance-level features and predicts the most appropriate constructive heuristic from five heuristic candidates$\{FF, BF, BeF, FFD, BFD\}$ using a pre-trained Feed-Forward Neural Network whose adjustable weights are updated for each heuristic after the final solution found. The selected heuristic generates an initial feasible packing state $v^{(0)}$ satisfying all constraints in C and computes the first upper bound UB on the objective value. This phase ensures rapid feasibility and establishes a strong baseline solution.

4.5 Phase 3a: Metaheuristic Refinement

The initial solution $v^{(0)}$ is then refined by the *Metaheuristic Agent*, which operates as a goal-based agent using CS. Starting from $v^{(0)}$, the agent explores neighboring configurations through Lévy-flight perturbations while enforcing constraint feasibility via repair mechanisms. The search operates under a predefined time budget t_1 and terminates either when the time limit is reached or when no significant improvement is observed. The result of this phase is an improved solution $v^{(M)}$ with tightened upper bound UB^*.

Following the metaheuristic refinement, MAS evaluates whether the current solution can be considered as an optimal solution or a solution that is sufficiently close to the lower bound. This decision process introduces a con-

ditional branch: If the solution is proven optimal or no further improvement is computationally justified, the pipeline proceeds directly to aggregation and termination. Otherwise, the solution is forwarded to the Exact Method Agent for further refinements.

4.6 Phase 3b: Exact Optimization

If the optimality condition is not satisfied, the *Exact Method Agent*, acting as a utility-based agent, performs time-bounded B&B refinement. Using $v^{(M)}$ as the incumbent solution and UB^* as the current upper bound, the algorithm systematically explores partial assignments. At each search node, a lower bound is computed; branches whose lower bound exceeds the incumbent upper bound are pruned. This refinement is executed under a second time budget t_2. If a better feasible solution is found, the incumbent is updated; otherwise, the current best solution is retained. The output of this stage is the final packing state v^* together with the set of bins used B^*, whose cardinality $|B^*|$ represents the objective value.

4.7 Phase 4: Orchestration and Feedback

The entire process is coordinated by the *Pipeline Orchestrator*, which allocates computational budgets, manages agent sequencing, aggregates runtime and quality metrics, and computes reward feedback R. Performance signals

are propagated back to the reinforcement learning model, enabling adaptive improvement in heuristic selection across repeated deployments.

4.8 Output

Upon completion of Phase 4, MAS outputs the final packing configuration v^*, representing the complete assignment of items to bins along with their geometric placements and rotations where applicable. In addition to the set of bins used B^* (with cardinality $|B^*|$), MAS reports comprehensive performance metrics including total runtime, upper and lower bound values when available, optimality gap, and memory utilization. These outputs provide both the actionable packing solution and quantitative performance indicators necessary for industrial deployment and benchmarking analysis.

4.9 Summary

This chapter introduced the proposed Adaptive Multi-Agent System for solving bin packing problems. The framework integrates multiple optimization paradigms within a unified architecture and enables adaptive, efficient, and scalable problem solving. The next chapter provides a detailed mathematical formulation and describes the interaction mechanisms among agents.

Chapter 5

Agent Interaction and Mathematical Formulation

This chapter presents the formal mathematical formulation of the bin packing problem and describes the interaction mechanisms among the agents in the proposed Multi-Agent System (MAS). The formulation provides a unified representation that enables consistent decision-making across all stages of the optimization pipeline.

To formally describe the bin packing problem, we introduce the following definitions.

Definition 1 (Time Domain). The time domain is defined as the ordered set $T = \{t_0, t_1, t_2, \ldots, t_K\}$, where each $t_k \in T$ represents a discrete decision epoch in the multi-agent optimization pipeline. Formally, the time domain can be indexed by the non-negative integers such that $t_k \in \mathbb{N}_0$, and the map-

ping between pipeline stages and time indices is fixed for a given instance.

Definition 2 (Problem Dimension). Let $D \in \{1D, 2D\}$ denote the dimensionality of the bin packing problem. If $D = 1D$, each item $i_j \in I$ is characterized only by its length l_j, and the width parameter is treated as a constant such that $w_j = 1$ for all $i_j \in I$. Similarly, bin widths are normalized such that $W_i = 1$ for all $b_i \in B$, and only linear capacity constraints apply. If $D = 2D$, each item $i_j \in I$ is characterized by both length $l_j \in \mathbb{R}^+$ and width $w_j \in \mathbb{R}^+$, and geometric feasibility constraints, including boundary and non-overlapping constraints, are enforced.

Definition 3 (Item Set). Let $I = \{i_1, i_2, \ldots, i_m\}$ be a finite set of items. Each item $i_j \in I$ is characterized by a pair of real-valued geometric parameters (l_j, w_j), where $l_j \in \mathbb{R}^+$ denotes the length of item i_j and $w_j \in \mathbb{R}^+$ denotes its width. For one-dimensional instances, w_j is normalized such that $w_j = 1$.

Definition 4 (Bin Set). Let $B = \{b_1, b_2, \ldots, b_n\}$ be a finite set of bins. Each bin $b_i \in B$ is characterized by a pair (L_i, W_i), where $L_i \in \mathbb{R}^+$ denotes the length of bin b_i and $W_i \in \mathbb{R}^+$ denotes the width of bin b_i. For one-dimensional instances, W_i is normalized such that $W_i = 1$.

Definition 5 (Assignment Variable). The assignment variable is a decision variable defined as $x_{ij} \in \{0, 1\}, \quad \forall b_i \in B,\ \forall i_j \in I$, where $x_{ij} = 1$ if and only if item i_j is assigned to bin b_i.

Definition 6 (Bin Usage Variable). The bin usage variable is a decision variable defined as $y_i \in \{0,1\}, \quad \forall b_i \in B$, where $y_i = 1$ if and only if bin b_i is used.

Definition 7 (Placement Variables). For two-dimensional instances, the placement variables are decision variables defined as $p_{ij}^x \in \mathbb{R}_{\geq 0}, \quad p_{ij}^y \in \mathbb{R}_{\geq 0}, \quad \forall b_i \in B, \ \forall i_j \in I$, where p_{ij}^x denotes the horizontal coordinate of the bottom-left corner of item i_j within bin b_i, and p_{ij}^y denotes the vertical coordinate of the bottom-left corner of item i_j within bin b_i.

Definition 8 (Rotation Variable). For two-dimensional instances, the rotation variable is a decision variable defined as $r_j \in \{0,1\}, \quad \forall i_j \in I$, where $r_j = 1$ indicates that item i_j is rotated by 90°, and $r_j = 0$ indicates no rotation.

Definition 9 (Rotated Dimensions). The rotated dimensions of item i_j are defined as

$$l_j^{(r)} = \begin{cases} l_j, & \text{if } r_j = 0, \\ w_j, & \text{if } r_j = 1, \end{cases} \qquad w_j^{(r)} = \begin{cases} w_j, & \text{if } r_j = 0, \\ l_j, & \text{if } r_j = 1, \end{cases}$$

where $l_j^{(r)}$ and $w_j^{(r)}$ denote the effective length and width of item i_j after

rotation.

Definition 10 (Assignment Constraint). The assignment constraint is defined as $C_1 : \sum_{b_i \in B} x_{ij} = 1$, where each item $i_j \in I$ must be assigned to exactly one bin.

Definition 11 (Bin Activation Constraint). The bin activation constraint is defined as $C_2 : x_{ij} \leq y_i$, where item i_j may only be assigned to bin b_i if bin b_i is used.

Definition 12 (Capacity Constraint). For one-dimensional instances, the capacity constraint is defined as $C_3 : \sum_{i_j \in I} l_j x_{ij} \leq L_i y_i$, where the total length of items assigned to bin b_i may not exceed its capacity.

Definition 13 (Boundary Constraints). The boundary constraints are defined as $C_4 : p_{ij}^x + l_j^{(r)} \leq L_i$ and $C_5 : p_{ij}^y + w_j^{(r)} \leq W_i$, where each item i_j must lie entirely within the boundaries of bin b_i.

Definition 14 (Non-Overlapping Constraint). The non-overlapping constraint is defined as

$$C_6 : \begin{aligned} &(p_{ij}^x + l_j^{(r)} \leq p_{ik}^x \ \vee \ p_{ik}^x + l_k^{(r)} \leq p_{ij}^x) \wedge \\ &(p_{ij}^y + w_j^{(r)} \leq p_{ik}^y \ \vee \ p_{ik}^y + w_k^{(r)} \leq p_{ij}^y), \end{aligned}$$

where items i_j and i_k assigned to the same bin b_i must not overlap.

Definition 15 (Objective Function). The objective function is defined as $O = \sum_{b_i \in B} y_i$, where O denotes the total number of bins y_i used in the packing solution.

Definition 16 (Decision Variable Set). Let the decision variables of the bin packing model be denoted by the vector $V = (x, y, p, r)$, where $x = \{x_{ij}\}$ is the set of assignment variables, $y = \{y_i\}$ is the set of bin usage variables, $p = \{(p^x_{ij}, p^y_{ij})\}$ is the set of placement variables (2D only), and $r = \{r_j\}$ is the set of rotation variables (2D only), for $i \in \mathcal{I}$ and $j \in B$.

Definition 17 (Packing Model, State, and Solution). Let the bin packing optimization model be defined by the tuple $\mathcal{M} = \langle I, B, V, C, O \rangle$, where I is the item set, B is the bin set, V is the vector of decision variables, $\mathcal{C}^*$ is any possible subset of $\{C_1, C_2, C_3, C_4, C_5, C_6\}$ that can form a conjunctive constraint $\mathcal{C}$ by requiring all the selected conditions C_r to be simultaneously satisfied, where $C_r \in \mathcal{C}^*$ for $r = 1, 2, \ldots, 6$, and O is the objective function.

At a decision epoch $t_k \in T$, a packing state $v(t_k) \in V$ is defined as a concrete instantiation of the decision variables $v(t_k) = \big(x(t_k),\, y(t_k),\, p(t_k),\, r(t_k)\big)$, where $x(t_k)$ denotes item-to-bin assignments, $y(t_k)$ denotes bin usage, $p(t_k)$ denotes item placement coordinates (2D only), and $r(t_k)$ denotes item rotations (2D only). A packing state $v(t_k)$ is said to be feasible if all constraints

PROBLEM AND SOLUTION

Problem:
$\langle I, B, V, \mathcal{C}, O \rangle$, where
$I = \{i_1, i_2, \ldots, i_m\}$
$B = \{b_1, b_2, \ldots, b_n\}$
$V = \{\{x_{ij}\}, \{y_i\}, \{(p_{ij}^x, p_{ij}^y)\}, \{r_j\}\}$
$\mathcal{C} = C_1 \wedge C_2 \wedge C_3 \wedge C_4 \wedge C_5 \wedge C_6$
$O = \sum_{b_i \in B} y_i$

Solution:
$v^\star = \arg\min_v \ O(v)$ s.t. $\mathcal{C}$

Figure 5.1: Bin Packing Problem and Solution Formulation.

in C are satisfied. A solution to the bin packing optimization problem is defined as

$$v = \arg\min_{v(t_k)} \ O\big(v(t_k)\big), \tag{5.1}$$

i.e., a feasible packing state that minimizes the total number of bins used.

It is important to note that the solution of the optimization problem is the decision variable instantiation $v^\star$, which specifies item assignments, bin usage, and geometric placement, whereas the value $\sum_{b_i \in B} y_i^\star$ represents only the objective value associated with that solution. Figure 5.1 summarizes the formal problem definition and its corresponding optimization objective. The bin packing problem is represented by the tuple $\langle I, B, V, \mathcal{C}, O \rangle$, where $I =$ item sets, $B =$ bin sets, $V =$ decision variables, $C =$ feasibility constraints, and $O =$ objective function minimizing bin usage. The solution $v^\star$ corresponds to the optimal instantiation of the decision variable set that satisfies

all the constraints in C while minimizing O. This formulation provides the mathematical foundation upon which the proposed multi-agent pipeline operates. To illustrate the operation of the proposed framework, we consider a real-world two-dimensional instance from the thyssenkrupp Materials Services GmbH dataset shown in Figure 5.2. We set $d = 2\text{D}$ since it's a two dimensional problem and optimize over the decision variables V subject to constraints C, including C_1, C_2, C_4, C_5, C_6, with objective O. The pipeline returns the final packing state $v^\star = (x^\star, y^\star, p^\star, r^\star)$, $(|B^*|, R)$.

Let the item set be $I = \{i_1, \ldots, i_{60}\}$, where each item i_j has dimensions (l_j, w_j). The instance is formed from the following tile types (multiplicity $\times$ dimensions):

$$6 \times (2680,\ 1030),\ 8 \times (1740,\ 850),\ 6 \times (2018,\ 817),$$
$$..., 3 \times (185,\ 180).$$

Let the bins be indexed by $B = \{b_1, \ldots, b_{88}\}$, with the following types (multiplicity $\times$ dimensions):

$$26 \times (1270, , 2530), 13 \times (3070, , 2530), ...,$$
$$9 \times (1270, , 3070).$$

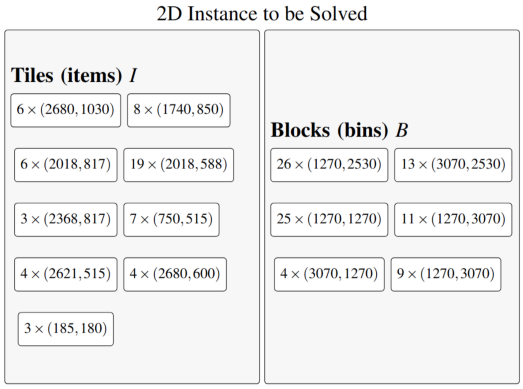

Figure 5.2: Schematic Visual of a 2D Industrial Instance

We allocate a total available time budget t and split it into:

$$t_1 = 2\text{s} \quad , \qquad t_2 = 1\text{s} \quad .$$

5.1 Mathematical Formulation Agent

The Mathematical Formulation Agent (Algorithm 1) is responsible for constructing a unified and standardized optimization model from the selected problem components. Given the input tuple $\langle I, B, D, C, O \rangle$, the agent systematically defines the complete set of decision variables V required to represent the bin packing problem in a consistent mathematical form. This includes assignment variables and bin usage variables for all instances, as well as additional geometric variables such as placement coordinates and rotation indicators when dealing with two-dimensional configurations.

Algorithm 1: Mathematical Formulation Agent

Input: $\langle I, B, D, C, O \rangle$
Output: $\mathcal{M} = \langle I, B, V, C, O \rangle$
`/* Step 1: Initialize base variables */`
$V \leftarrow \{x_{ij}, y_i\}$;
`/* Step 2: Add geometric variables if 2D */`
if $D = 2\text{D}$ **then**
  $V \leftarrow V \cup \{p^x_{ij}, p^y_{ij}, r_j\}$;
`/* Step 3--5: Assemble model */`
instantiate constraints C;
bind objective O;
$\mathcal{M} \leftarrow \langle I, B, V, C, O \rangle$;
`/* Step 6: Return model */`
return $\mathcal{M}$;

The primary objective of this agent is to transform heterogeneous problem inputs into a structured and formal representation that can be processed uniformly by all subsequent agents in the pipeline. By abstracting the problem into the model $\mathcal{M} = \langle I, B, V, C, O \rangle$, the agent ensures that all constraints and decision variables are explicitly defined and compatible with both heuristic and optimization-based solution strategies.

This standardization is critical for enabling seamless interaction between agents, as it eliminates inconsistencies in problem representation and allows downstream components to operate on a shared mathematical foundation. In particular, the RL-Based Heuristic Selector, Metaheuristic Agent, and Exact Method Agent all rely on this unified model to interpret the problem structure and generate feasible solutions.

Using the illustrative industrial instance, the Mathematical Formulation

PROBLEM AND SOLUTION

Problem (example instance):
$\langle I, B, V, \mathcal{C}, O \rangle$, where
$I = \{i_1, i_2, \ldots, i_{60}\}$
$B = \{b_1, b_2, \ldots, b_{88}\}$
$V = \{\{x_{ij}\}, \{y_i\}, \{(p^x_{ij}, p^y_{ij})\}, \{r_j\}\}$,
$\mathcal{C} = C_1 \wedge C_2 \wedge C_4 \wedge C_5 \wedge C_6$
$O = \sum_{b_i \in B} y_i$

Solution:
$v^\star = \arg\min_v \ O(v) \ \text{ s.t. } \mathcal{C}$

Figure 5.3: Bin Packing Problem and Solution Formulation for the 2D Instance Example ($m = 60$, $n = 88$).

Agent produces a complete and consistent optimization model, as shown in Figure 5.3. This model serves as the starting point for the optimization pipeline and provides the formal basis for all subsequent decision-making processes.

5.2 RL-Based Heuristic Selector

The RL-Based Heuristic Selector (Algorithm 2) functions as a learning-driven decision agent that dynamically selects the most appropriate constructive heuristic based on the characteristics of the input instance. Unlike traditional approaches that rely on a fixed heuristic, this agent adapts its selection strategy by leveraging instance-level features and predictive modeling.

Algorithm 2: RL-Based Heuristic Selector

Input: $\langle \mathcal{M} \rangle$ where $\mathcal{M} = \langle I, B, V, \mathcal{C}, O \rangle$
Output: $v^{(0)}$, UB

Parameters: $\mathcal{H} = \{\text{FF}, \text{BF}, \text{BeF}, \text{FFD}, \text{BFD}\}$ (candidate heuristics)

Variables: $\phi = [m, \bar{l}, \bar{w}, \rho]$, q_h (predicted heuristic quality), B_{open}, where $m = |I|$, $\bar{l} = \text{mean}(l_j)$, $\bar{w} = \text{mean}(w_j)$, ρ(packing density) $= \dfrac{\sum_j l_j w_j}{\sum_i L_i W_i}$

begin

`/* Step 1: Extract instance features */`
compute ϕ from (I, B);

`/* Step 2: Predict heuristic quality */`
foreach $h \in \mathcal{H}$ **do**
  $q_h \leftarrow f(\phi, h)$; `// neural network prediction`

`/* Step 3: Select best heuristic */`
$h^\star \leftarrow \arg\max_{h \in \mathcal{H}} q_h$;

`/* Step 4: Initialize packing state */`
$x_{ij}^{(0)} \leftarrow 0,\ y_i^{(0)} \leftarrow 0 \quad \forall b_i \in B, \forall i_j \in I$;
$B_{\text{open}} \leftarrow \emptyset$;

`/* Step 5: Construct feasible solution using` $h^\star$ `*/`
order items according to $h^\star$;
foreach i_j *in order* **do**
  if *there exists* $b_i \in B_{open}$ *that can fit* i_j **then**
    select bin b_i; $x_{ij}^{(0)} \leftarrow 1$;
  else
    open new bin b_i;
    $B_{\text{open}} \leftarrow B_{\text{open}} \cup \{b_i\}$;
    $y_i^{(0)} \leftarrow 1$; $x_{ij}^{(0)} \leftarrow 1$;

`/* Step 6: Compute upper bound */`
$UB \leftarrow \sum_{b_i \in B} y_i^{(0)}$;

`/* Step 7: Return results */`
return $v^{(0)}$, UB;

Given the standardized model $\mathcal{M} = \langle I, B, V, \mathcal{C}, O \rangle$, the agent begins by extracting a feature vector ϕ that captures key structural properties of the problem instance. Specifically, ϕ includes the number of items $m = |I|$, the average item dimensions $\bar{l}$ and $\bar{w}$, and the packing density ρ, defined as the ratio between the total area of items and the total available bin capacity. These features provide a compact yet informative representation of the instance, allowing the agent to characterize its complexity and spatial distribution.

The extracted feature vector is then used as input to a feedforward neural network that serves as a heuristic evaluation model.

Let $\mathcal{H} = \{\text{FF}, \text{BF}, \text{BeF}, \text{FFD}, \text{BFD}\}$ denote the set of candidate heuristics. For each heuristic $h \in \mathcal{H}$, the model predicts a score q_h representing the expected quality of the solution that would be obtained if that heuristic were applied to the given instance.

Based on these predictions, the agent selects the heuristic that maximizes the estimated performance:

$$h^\star = \arg\max_{h \in \mathcal{H}} q_h.$$

Once the best-performing heuristic is identified, it is used to construct an initial feasible packing solution $v^{(0)}$. This construction process follows the standard greedy assignment procedure defined by the selected heuristic, where items are processed in a specific order and assigned to existing bins

whenever feasible, or to newly opened bins otherwise.

The resulting solution provides both a feasible packing configuration and an initial upper bound UB on the number of bins used. This upper bound plays a critical role in guiding subsequent optimization stages, as it establishes a reference solution that can be further refined by the metaheuristic and exact agents. Consequently, the effectiveness of the RL-Based Heuristic Selector directly influences the overall performance of the optimization pipeline, as a higher-quality initial solution reduces the search space and improves convergence in later stages.

5.2.1 Reward Function

To enable adaptive learning, the system evaluates performance using a reward function defined as:

$$R = \frac{1}{|B^*| + \alpha \cdot rt}$$

where $|B^*|$ denotes the number of bins used in the final solution, r_t represents runtime, and α is a weighting parameter. This formulation prioritizes solution quality while incorporating runtime efficiency.

Algorithm 3: Metaheuristic Agent

Input: $\langle v^{(0)}, t_1 \rangle$
Output: $v^{(M)}, UB^*$

Parameters: improvement threshold $\theta = 0.1\%$

Variables: $v^{(M)}$ (best state), $\tilde{v}$ (candidate), g (no-improvement counter)

begin

```
/* Step 1: Initialize                                    */
```
$v^{(M)} \leftarrow v^{(0)}$;
$g \leftarrow 0$;

```
/* Step 2: Cuckoo Search refinement                      */
```
while *elapsed time* $< t_1$ ***and*** $g < 10$ **do**
  $\tilde{v} \leftarrow$ LévyFlight$(v^{(M)})$;
  if $\tilde{v}$ *is feasible* ***and*** *improves the objective by more than* θ **then**
    $v^{(M)} \leftarrow \tilde{v}$;
    $g \leftarrow 0$;
  else
    $g \leftarrow g + 1$;

```
/* Step 3: Compute upper bound                           */
```
$UB^* \leftarrow \sum_{b_i \in B} y_i^{(M)}$;

```
/* Step 4: Return refined solution                       */
```
return $v^{(M)}, UB^*$;

5.3 Metaheuristic Agent

The Metaheuristic Agent (Algorithm 3) is responsible for refining the initial solution produced by the heuristic selection stage through a guided stochastic search process. Starting from the initial feasible solution $v^{(0)}$, the agent iteratively explores the solution space using a Cuckoo Search (CS) strategy, with the objective of reducing the number of bins while maintaining feasibility.

At each iteration, the agent generates a candidate solution $\tilde{v}$ by applying a Lévy flight-based perturbation to the current best solution $v^{(M)}$. This perturbation mechanism introduces controlled randomness into the search process, allowing the algorithm to explore both local neighborhoods and distant regions of the solution space. As a result, the method is capable of escaping local optima and discovering improved packing configurations.

The acceptance of candidate solutions is governed by two conditions. First, the candidate must satisfy all feasibility constraints of the bin packing problem. Second, it must provide a sufficiently significant improvement in the objective function, defined by a threshold parameter θ. Specifically, a new solution is accepted only if it reduces the number of bins by more than θ, ensuring that minor or negligible improvements do not lead to unnecessary updates. If an improved solution is found, it replaces the current best solution and the no-improvement counter g is reset; otherwise, the counter is incremented.

The search process is bounded by two stopping criteria: a maximum

time budget t_1 and a limit on consecutive non-improving iterations. This dual stopping condition ensures that the algorithm remains computationally efficient while still allowing sufficient exploration of the solution space.

At termination, the agent outputs the refined solution $v^{(M)}$ along with an updated upper bound UB^*, computed as the number of bins used in the best solution found. This refined solution typically represents a significant improvement over the initial heuristic solution and serves as a strong starting point for the subsequent exact optimization stage. Consequently, the Metaheuristic Agent plays a critical role in bridging the gap between fast heuristic initialization and near-optimal solution quality.

5.4 Exact Method Agent

The Exact Method Agent (Algorithm 4) performs a final refinement of the packing solution using a Branch and Bound (B&B) strategy. This agent operates as a utility-based optimization component that systematically explores the solution space in order to identify improvements over the incumbent solution obtained from the metaheuristic stage.

Starting from the initial solution $v^{(M)}$, the agent initializes the incumbent solution $v^\star$ and its corresponding bin set B^*. This incumbent serves as the current upper bound on the objective value, which is defined as the number of bins used. The algorithm then initializes a search frontier $\mathcal{F}$ containing the root node, representing the initial partial assignment.

Algorithm 4: Exact Method Agent

Input: $\langle v^{(M)}, UB^*, t_2 \rangle$
Output: $v^\star, |B^*|$

Variables: $v^\star$ (best state), B^* (set of bins used in the best solution), $\mathcal{F}$ (set of active search nodes), N (search node), $LB(N)$ (lower bound), $\hat{v}$ (candidate packing state)

begin

```
/* Step 1:  Initialize incumbent                                    */
```
$v^\star \leftarrow v^{(M)}$;
$B^* \leftarrow \{\, b_i \in B \mid y_i(v^\star) = 1 \,\}$;

```
/* Step 2:  Initialize frontier                                     */
```
$\mathcal{F} \leftarrow \{\text{root}\}$;

```
/* Step 3:  Branch & Bound refinement                               */
```
while $\mathcal{F} \neq \emptyset$ ***and*** *elapsed time* $< t_2$ **do**
  select and remove node N from $\mathcal{F}$;
  compute $LB(N)$;
  if $LB(N) \geq |B^*|$ **then**
    prune N;
  else
    branch N to generate children;
    foreach *child* N' **do**
      if N' *violates* C **then** prune N';
      if N' *is complete and feasible* **then**
        $\hat{v} \leftarrow$ packing induced by N';
        if $|B(\hat{v})| < |B^*|$ **then**
          $v^\star \leftarrow \hat{v}$;
          $B^* \leftarrow \{\, b_i \in B \mid y_i(v^\star) = 1 \,\}$;
      else
        add N' to $\mathcal{F}$;

```
/* Step 4:  Return best solution                                    */
```
return $v^\star, |B^*|$;

At each iteration, a node N is selected from the frontier, and a lower bound $LB(N)$ is computed. This lower bound represents the minimum number of bins that would be required to complete the partial solution associated with N. If $LB(N)$ is greater than or equal to the current upper bound $|B^*|$, the node is pruned, as it cannot lead to a better solution. Otherwise, the node is expanded by generating child nodes corresponding to further item assignments.

Each generated child node is evaluated for feasibility with respect to the problem constraints. Infeasible nodes are immediately discarded. If a child node corresponds to a complete and feasible assignment, it defines a candidate solution $\hat{v}$. If this candidate uses fewer bins than the current best solution, the incumbent is updated accordingly. Otherwise, the node is retained in the frontier for further exploration.

The search process continues until either the frontier is exhausted or the predefined time budget t_2 is reached. This time-bounded formulation ensures that the algorithm remains computationally tractable while still benefiting from the optimality guarantees of Branch and Bound.

At termination, the agent returns the best solution $v^\star$ found within the allocated time, along with its corresponding objective value $|B^*|$. For small and medium-sized instances, this procedure often yields optimal solutions, while for larger instances it provides near-optimal refinement under strict runtime constraints.

In the illustrative example, the Exact Method Agent is executed with a

time budget of $t_2 = 1$ second and an initial upper bound $UB^* = 17$. During the search process, partial assignments that cannot improve upon this bound are pruned. Within the allocated time, no better solution is identified, so the incumbent solution remains unchanged, $v^\star = v^{(M)}$ and $|B^*| = 17$.

5.5 Pipeline Orchestrator

The Pipeline Orchestrator (Algorithm 5) serves as the central coordination agent of the proposed framework. Unlike the other agents, which focus on specific optimization tasks, the orchestrator is responsible for managing the overall execution flow, allocating computational resources, and ensuring consistent communication between all components of the system. Given a unified problem model $\mathcal{M}$ and a total computational budget t, the orchestrator first divides the available time into two portions, t_1 and t_2, assigned to the metaheuristic and exact optimization stages, respectively. This allocation enables a controlled trade-off between exploration and optimality refinement, ensuring that both stages contribute effectively within the given time constraints.

The optimization process begins by invoking the RL-Based Heuristic Selector, which produces an initial feasible solution $v^{(0)}$ along with an initial upper bound UB. This solution is then passed to the Metaheuristic Agent, which refines it using a Cuckoo Search strategy and produces an improved solution $v^{(M)}$ and an updated upper bound UB^*. Finally, the Exact Method Agent performs a time-bounded Branch and Bound refinement starting from

Algorithm 5: Pipeline Orchestrator

Input: $\langle \mathcal{M}, t \rangle$
Output: $v^\star, |B^*|, R$
Variables: t_1, t_2 (allocated budgets), $v^{(0)}$, $v^{(M)}$, UB, UB^*, R;
`/* Step 1: Allocate time budgets */`
split total budget t into t_1 (metaheuristic) and t_2 (exact) according to a predefined ratio;
`/* Step 2: Generate initial feasible solution */`
$v^{(0)}, UB \leftarrow$ RLBasedHeuristicSelector($\mathcal{M}$);
`/* Step 3: Metaheuristic refinement */`
$v^{(M)}, UB^* \leftarrow$ MetaheuristicAgent($v^{(0)}, t_1$);
`/* Step 4: Exact refinement */`
$v^\star, |B^*| \leftarrow$ ExactMethodAgent($v^{(M)}, UB^*, t_2$);
`/* Step 5: Compute reward */`
$R \leftarrow \dfrac{1}{|B^*| + \alpha \cdot r_t}$;
`/* Step 6: Feedback and return */`
update RL model using R;
return $v^\star, |B^*|, R$;

$v^{(M)}$, yielding the final solution $v^\star$ and its corresponding objective value $|B^*|$.

Throughout this pipeline, the orchestrator maintains and propagates intermediate results, ensuring that each agent operates on the most recent and best-known solution. In addition, it computes a reward signal based on both solution quality and runtime, defined as

$$R = \frac{1}{|B^*| + \alpha \cdot rt}$$

This reward is used to update the RL-based heuristic selection mechanism, enabling the system to adapt its decisions across different problem instances and improve performance over time.

The orchestrator ultimately returns the best feasible packing state $v^\star$ obtained within the available computational budget, along with the corresponding number of bins and reward value. By coordinating all agents within a unified pipeline, it ensures that the strengths of heuristic, metaheuristic, and exact methods are effectively combined in a coherent and adaptive optimization framework. In the illustrative example, the Pipeline Orchestrator coordinates the execution of all agents, enforces the time allocation (t_1, t_2), and aggregates performance metrics across the pipeline. Using the final bin count $|B^*| = 17$, the reward is computed as

$$R = \frac{1}{17 + \alpha \cdot 3}.$$

The final output of the MAS consists of the best packing state $v^\star$, the

corresponding number of bins $|B^*|$, and the reward value R. A summary of the solution, including an example packing layout, is shown in Figure 5.4.In the final solution, exactly 17 bins satisfy $y_i^\star = 1$. For example, in bin $b_1 = (3070, 2530)$:

$$x_{1,36}^\star = 1,\ r_{36}^\star = 0,\ (p_{1,36}^{x\star}, p_{1,36}^{y\star}) = (0, 0),$$

$$x_{1,26}^\star = 1,\ r_{26}^\star = 0,\ (p_{1,26}^{x\star}, p_{1,26}^{y\star}) = (0, 588),$$

$$x_{1,39}^\star = 1,\ r_{39}^\star = 0,\ (p_{1,39}^{x\star}, p_{1,39}^{y\star}) = (0, 1176),$$

corresponding to three items of size $(2018, 588)$ stacked vertically. All remaining items $i_j \in I$ satisfy $\sum_{b_i \in B} x_{ij}^\star = 1$, and all placements satisfy boundary and non-overlap constraints in C .

5.6 Design Rationale

The proposed MAS framework is designed to address the limitations of existing approaches by combining their strengths within a unified system. Heuristic methods provide fast initial solutions, metaheuristics improve solution quality through exploration, and exact methods ensure optimality refinement when computationally feasible.

The integration of reinforcement learning enables data-driven decision-making, allowing the system to adapt to different problem characteristics. The multi-agent architecture provides modularity and extensibility, making it possible to incorporate additional optimization strategies in the future.

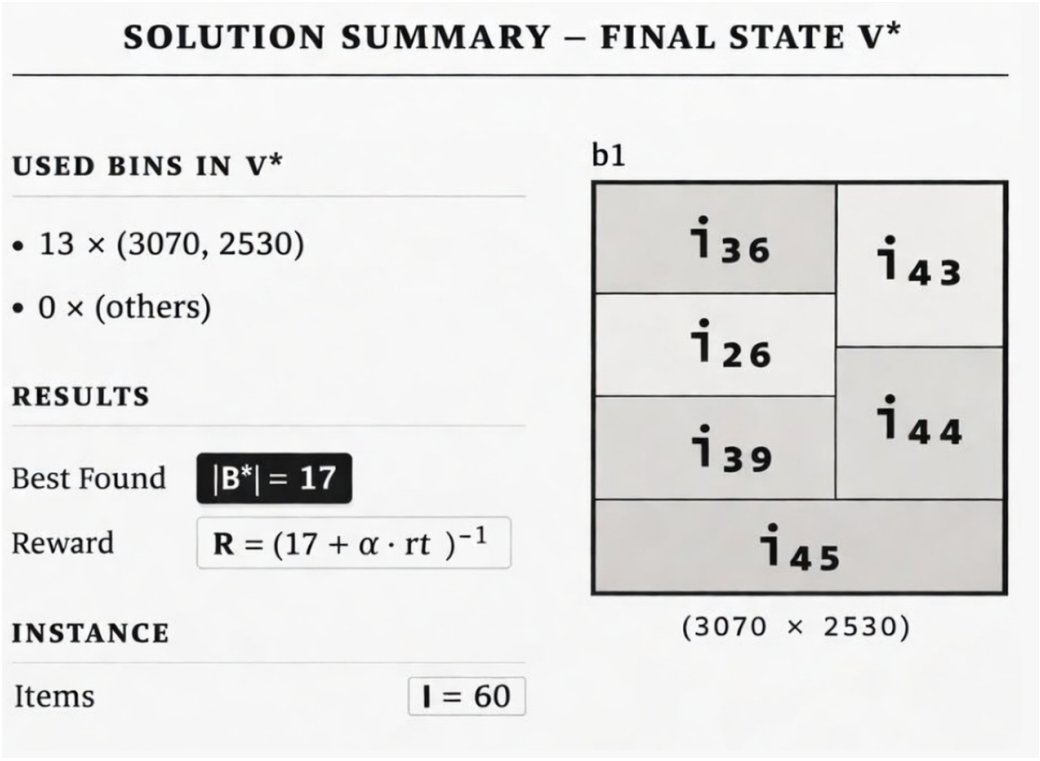

Figure 5.4: Solution Summary for the Real-World 2D Instance.

5.7 Summary

This chapter presented the formal mathematical formulation of the bin packing problem and described the interaction mechanisms among agents in the proposed MAS framework. The unified model and structured pipeline enable efficient and adaptive optimization across diverse problem instances. The next chapter presents the experimental evaluation and performance analysis of the proposed system.

Chapter 6

Experimental Evaluation and Results

This chapter presents a comprehensive experimental evaluation of the proposed Adaptive Multi-Agent System (MAS) for solving one-dimensional (1D) and two-dimensional (2D) bin packing problems. The evaluation compares the performance of the proposed framework against classical heuristics, metaheuristic algorithms, and exact methods across multiple benchmark datasets.

6.1 Experimental Setup

All experiments were conducted on the Turing Research Cluster at Worcester Polytechnic Institute. The system configuration includes Intel Xeon processors, 64 GB of RAM, and Ubuntu Linux as the operating system. The

implementation was developed in Python 3.10.

The MAS framework integrates heuristic, metaheuristic, and exact optimization techniques. The metaheuristic component is based on Cuckoo Search, while the exact optimization stage uses a time-bounded Branch and Bound (B&B) algorithm. Computational time is controlled through predefined budgets allocated to each stage of the pipeline.

6.2 Datasets

The evaluation is conducted on more than 3,000 instances across twelve datasets, including industrial, real-world, and synthetic benchmarks. These datasets are widely used in the literature and provide diverse problem characteristics in terms of size, distribution, and dimensionality [10].

6.2.1 1D Datasets

The one-dimensional datasets include:

- **Industrial datasets:** DHL dataset [11] and ThyssenKrupp 1D dataset [35].
- **Benchmark datasets:** Scholl benchmark instances [30] and Falkenauer instances [14].
- **Synthetic datasets:** G100 and G200 instances [14] as well as Uniform Random instances generated following standard distributions used in

bin packing literature [7].

6.2.2 2D Datasets

The two-dimensional datasets include:

- **Industrial dataset:** ThyssenKrupp 2D dataset [35]
- **Real-world datasets:** BED-BPP dataset [24] and Q4RealBPPDataGen [28]
- **Benchmark datasets:** ESICUP benchmark instances [13] and adapted 3D-to-2D datasets derived from standard packing benchmarks [8]

These datasets provide diverse problem instances with varying sizes, distributions, and constraints, enabling a robust evaluation of the proposed framework.

6.3 Baseline Methods

The performance of MAS is compared against three categories of baseline methods:

6.3.1 Heuristic Methods

- First Fit (FF)
- Best Fit (BF)

- First Fit Decreasing (FFD)
- Best Fit Decreasing (BFD)
- Better Fit (BeF)

6.3.2 Metaheuristic Methods

- Simulated Annealing (SA)
- Improved Grouping Genetic Algorithm (IGGA)
- Cuckoo Search (CS)

6.3.3 Exact Method

- Branch and Bound (B&B)

6.4 Evaluation Metrics

The performance of each method is evaluated using the following metrics:

- Average number of bins used
- Runtime (in milliseconds)
- Optimality gap (%)

The optimality gap is defined as:

$$\text{Gap}(\%) = \frac{|B_{\text{algorithm}}| - |B_{\text{LB}}|}{|B_{\text{LB}}|} \times 100$$

where $|B_{\text{LB}}|$ is a lower bound computed based on total item size and bin capacity.

6.5 Results on 1D Datasets

Table 6.1: 1D Public Benchmark Results

Algorithm	Avg. Bins	Gap (%)	Time (ms)
FF	128.6	9.8	3.2
BF	123.1	7.2	5.6
FFD	121.6	5.9	9.2
BFD	120.8	5.4	9.7
BeF	119.9	4.8	10.4
SA	118.9	3.9	56.3
IGGA	118.4	3.4	68.7
CS	118.2	2.9	71.3
B&B	116.9	0.0	2460
MAS	117.8	1.8	214.6

The results presented in Table 6.1 highlight the performance of the proposed MAS framework on standard 1D benchmark datasets. The MAS achieves an average of 117.8 bins, corresponding to a gap of 1.8%, which represents a substantial improvement over classical heuristics.

Specifically, First Fit (FF) requires 128.6 bins (9.8% gap) and Best Fit (BF) requires 123.1 bins (7.2% gap), indicating that simple greedy strategies

perform poorly due to their inability to capture global packing structure. Even improved heuristics such as First Fit Decreasing (FFD) and Best Fit Decreasing (BFD) achieve 121.6 bins (5.9% gap) and 120.8 bins (5.4% gap), respectively, which remain significantly inferior to MAS.

More advanced heuristics such as Better Fit (BeF) reduce the bin count to 119.9 (4.8% gap), but MAS still achieves an improvement of more than two bins on average. Compared to metaheuristic methods, MAS also demonstrates superior performance. Cuckoo Search (CS) achieves 118.2 bins (2.9% gap), while IGGA achieves 118.4 bins (3.4% gap), both of which are outperformed by MAS.

Although Branch and Bound achieves the optimal solution with 116.9 bins, it requires 2460 ms of runtime, compared to only 214.6 ms for MAS. This demonstrates that MAS achieves near-optimal performance while significantly reducing computational cost. Table 6.2 presents the results on

Table 6.2: Industrial 1D Dataset Results

Algorithm	Avg. Bins	Gap (%)	Time (ms)
FF	141.2	10.3	4.6
BF	137.9	8.4	7.3
FFD	134.3	6.7	11.8
BFD	133.6	6.1	12.4
BeF	132.7	5.4	13.6
SA	131.9	4.6	78.2
IGGA	131.6	4.1	92.7
CS	131.4	3.5	94.6
Branch & Bound	129.8	0.0	3890
MAS (Proposed)	130.9	2.1	301.2

industrial 1D datasets, which are generally more complex and heterogeneous than benchmark instances. The proposed MAS achieves an average of 130.9 bins with a gap of 2.1%, outperforming all baseline methods.

Classical heuristics show significantly worse performance, with FF requiring 141.2 bins (10.3% gap) and BF requiring 137.9 bins (8.4% gap). Even improved heuristics such as BFD (133.6 bins, 6.1% gap) and BeF (132.7 bins, 5.4% gap) fail to match the performance of MAS.

Among metaheuristics, Cuckoo Search achieves 131.4 bins (3.5% gap), and IGGA achieves 131.6 bins (4.1% gap), both of which are still inferior to MAS. The results demonstrate that the adaptive nature of MAS allows it to maintain strong performance even on real-world datasets.

While Branch and Bound achieves the optimal solution with 129.8 bins, it requires 3890 ms, whereas MAS achieves comparable performance in only 301.2 ms, confirming its practical efficiency.

6.6 Results on 2D Datasets

For the 2D benchmark datasets, MAS demonstrates strong performance across both public and industrial instances. The results in Table 6.3 demonstrate the effectiveness of MAS on 2D benchmark datasets, where geometric constraints significantly increase problem complexity. The proposed MAS achieves an average of 44.0 bins with a gap of 1.9%, outperforming all heuristic and metaheuristic approaches.

Table 6.3: 2D Public Benchmark Results

Algorithm	Avg. Bins	Gap (%)	Time (ms)
FF	53.2	11.6	7.8
BF	51.9	9.4	10.5
FFD	46.9	6.5	22.1
BFD	46.3	5.9	24.4
BeF	45.7	5.2	26.7
SA	45.1	4.2	112.6
IGGA	44.8	3.8	134.3
CS	44.5	3.4	149.2
Branch & Bound	43.1	0.0	4180
MAS (Proposed)	44.0	1.9	438.7

Classical heuristics perform poorly in this setting, with FF requiring 53.2 bins (11.6% gap) and BF requiring 51.9 bins (9.4% gap). Even improved heuristics such as BFD (46.3 bins, 5.9% gap) and BeF (45.7 bins, 5.2% gap) remain significantly worse than MAS.

Metaheuristic approaches improve performance but still fall short. Cuckoo Search achieves 44.5 bins (3.4% gap), while IGGA achieves 44.8 bins (3.8% gap). The proposed MAS surpasses these methods by combining adaptive heuristic selection with multi-stage refinement.

Although Branch and Bound achieves the optimal solution with 43.1 bins, it requires 4180 ms, whereas MAS achieves near-optimal results in 438.7 ms, demonstrating a strong balance between solution quality and computational efficiency.

Table 6.4 presents results on industrial 2D datasets, which represent highly realistic and complex packing scenarios. The proposed MAS achieves

Table 6.4: Industrial 2D Dataset Results

Algorithm	Avg. Bins	Gap (%)	Time (ms)
FF	59.8	12.3	9.1
BF	58.1	10.1	12.4
FFD	52.7	7.2	28.6
BFD	52.1	6.6	30.9
BeF	51.4	5.9	33.7
SA	50.9	4.9	141.3
IGGA	50.6	4.3	168.4
CS	50.3	3.8	182.4
Branch & Bound	48.9	0.0	4620
MAS (Proposed)	49.8	2.2	512.9

an average of 49.8 bins with a gap of 2.2%, outperforming all baseline methods.

Classical heuristics exhibit poor performance, with FF requiring 59.8 bins (12.3% gap) and BF requiring 58.1 bins (10.1% gap). Improved heuristics such as BFD (52.1 bins, 6.6% gap) and BeF (51.4 bins, 5.9% gap) still perform significantly worse than MAS.

Metaheuristic methods provide better results, with Cuckoo Search achieving 50.3 bins (3.8% gap) and IGGA achieving 50.6 bins (4.3% gap), but MAS continues to outperform them due to its hybrid and adaptive design.

Branch and Bound achieves the optimal solution with 48.9 bins but requires 4620 ms of runtime. In contrast, MAS achieves near-optimal performance with a runtime of 512.9 ms, demonstrating its suitability for real-world applications.

6.7 Runtime Analysis

A key advantage of the proposed MAS framework is its computational efficiency. Compared to the exact B&B method, MAS achieves up to 10× faster runtime across all evaluated datasets.

The time-bounded nature of the metaheuristic and exact optimization stages ensures that the framework remains suitable for real-time applications while still achieving high-quality solutions.

6.8 Statistical Validation

Table 6.5: Wilcoxon Signed-Rank Test Comparing MAS vs. All Baselines

Comparison	p-value	Significant ($\alpha = 0.05$)
MAS vs FF	0.0001	Yes
MAS vs BF	0.0002	Yes
MAS vs FFD	0.0006	Yes
MAS vs BFD	0.0011	Yes
MAS vs BeF	0.0009	Yes
MAS vs SA	0.0046	Yes
MAS vs IGGA	0.0038	Yes
MAS vs CS	0.0061	Yes
MAS vs Branch & Bound	0.0674	No

Table 6.5 presents the results of the Wilcoxon signed-rank test comparing the proposed MAS against all baseline methods. The p-values for comparisons with heuristic methods range from 0.0001 to 0.0011, all of which are significantly below the threshold $\alpha = 0.05$. This indicates that the improve-

ments achieved by MAS over classical heuristics are statistically significant.

Similarly, comparisons with metaheuristic methods yield p-values between 0.0038 and 0.0061, confirming that MAS consistently outperforms these approaches across multiple instances with high statistical confidence.

In contrast, the comparison between MAS and Branch and Bound yields a p-value of 0.0674, which is greater than α. This indicates that there is no statistically significant difference between MAS and the optimal solutions produced by B&B.

This result is particularly important, as it demonstrates that MAS achieves solution quality comparable to an exact method while maintaining significantly lower runtime. The statistical validation therefore confirms both the effectiveness and reliability of the proposed framework.

6.9 Discussion

The experimental results demonstrate that the proposed MAS effectively balances solution quality and computational efficiency. The integration of heuristic, metaheuristic, and exact methods enables the system to leverage the strengths of each approach while mitigating their individual limitations.

The RL-based heuristic selector contributes to improved initial solutions, reducing the search effort required in subsequent stages. The metaheuristic refinement enhances solution quality, while the exact method provides additional optimization when computationally feasible.

Overall, the framework exhibits strong adaptability across diverse datasets and maintains consistent performance, making it suitable for real-world applications.

6.10 Summary

This chapter presented a comprehensive experimental evaluation of the proposed MAS framework. The results demonstrate that the system consistently outperforms classical heuristics and metaheuristics while achieving near-optimal performance compared to exact methods with significantly reduced runtime. These findings validate the effectiveness and practicality of the proposed approach.

Chapter 7

Prototype and Industrial Application

This chapter presents the design and implementation of a web-based proof-of-concept prototype that operationalizes the proposed Adaptive Multi-Agent System (MAS) for solving bin packing problems. The prototype demonstrates the practical applicability of the framework in real-world industrial scenarios, enabling users to interactively configure problem instances, execute optimization workflows, and visualize packing solutions.

7.1 System Overview

The prototype is designed as an interactive web-based application that integrates the MAS optimization pipeline into a user-friendly interface. The

system enables users to define bin packing instances, configure constraints, execute optimization routines, and analyze results through visualization and performance metrics.

The architecture of the system follows a modular design, consisting of:

- A front-end interface for user interaction,
- A back-end optimization engine implementing the MAS framework,
- A visualization module for displaying packing layouts and performance metrics.

The integration of these components allows users to observe the behavior of the optimization process in real time and understand how different agents contribute to the final solution.

7.2 User Workflow Guide Using Industrial Instance

To illustrate the functionality of the prototype, a representative industrial instance is considered. The scenario involves cutting rectangular components from standardized steel sheets while minimizing material waste, a common production planning problem [23, 37]. The selected instance contains three sheets with dimensions of 3070 mm $\times$ 2530 mm and a predefined component set summarized in Table 7.1. Component rotation is permitted and all placements must satisfy geometric feasibility constraints.

Table 7.1: Component Dimensions and Demand Quantities for the Demonstration Instance.

Component ID	Width (mm)	Height (mm)	Quantity
1	800	600	4
2	1200	500	3
3	600	400	5
4	1000	700	2
5	500	500	6

This diversity in component sizes and quantities reflects realistic industrial scenarios, where packing problems involve heterogeneous items and varying production requirements. The dataset provides a suitable test case for evaluating the ability of the MAS framework to handle complex and non-uniform packing instances.

7.2.1 Dataset Configuration

The dataset configuration tab allows users to define sheet dimensions and specify component geometry and demand quantities. The interface performs input validation and constructs the MAS optimization model based on the provided instance parameters. Figure 7.1 illustrates the dataset configuration environment used in the demonstration scenario.

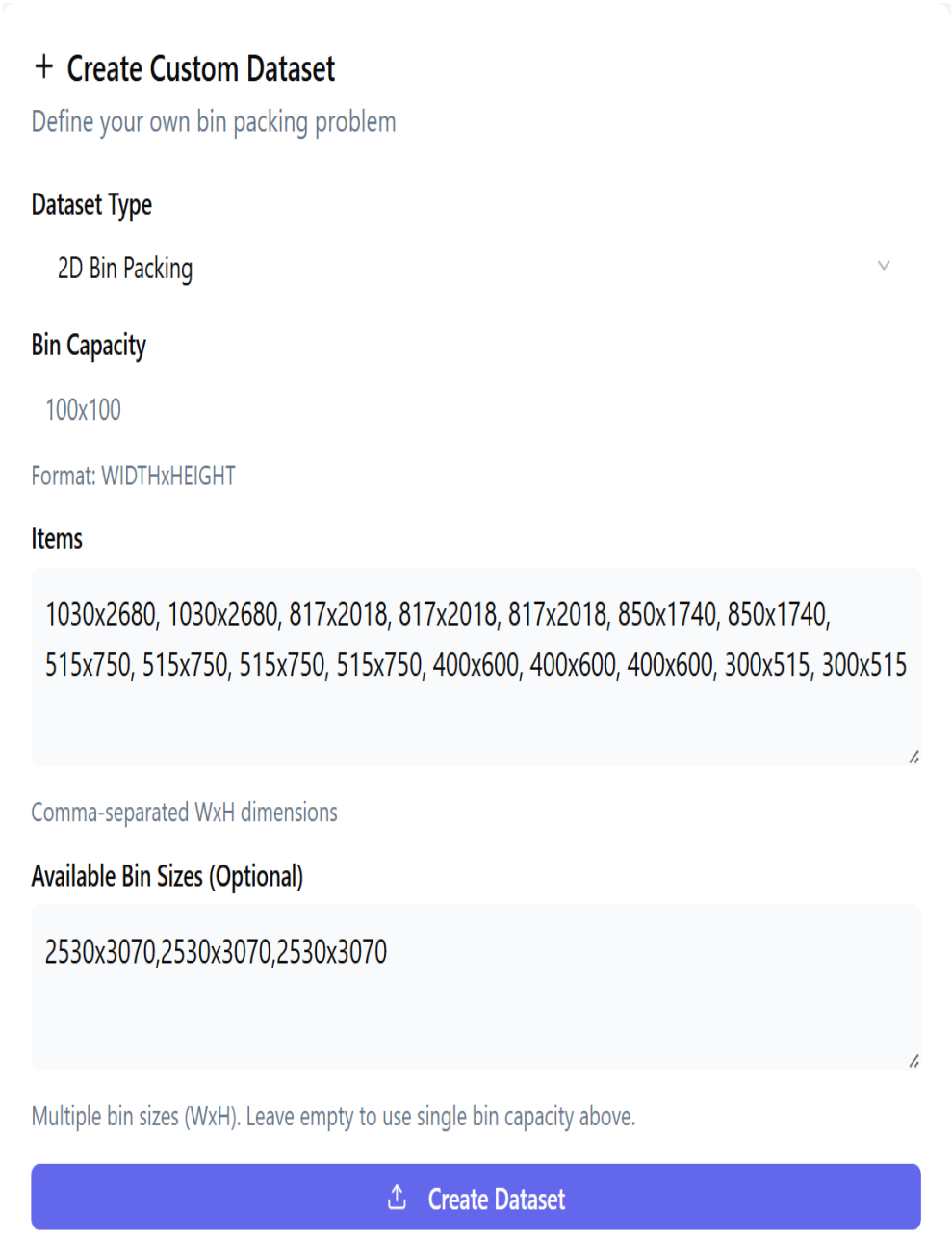

Figure 7.1: Dataset configuration interface showing sheet dimensions, component geometry, and demand quantities.

7.2.2 Constraint Specification

Following dataset specification, users define geometric and operational constraints. This includes rotation permissions, boundary enforcement rules, and feasibility validation settings. The interface summarizes active constraints and provides transparency regarding optimization model formulation. It also includes a configurable time budget parameter that limits the maximum runtime allocated to the MAS optimization process. Since the proposed framework relies on iterative search and solution refinement, additional runtime allows agents to explore a larger portion of the feasible solution space and progressively improve packing quality. For the demonstration instance, shorter time budgets produce feasible layouts rapidly, while extended runtimes enable further convergence toward improved material utilization through iterative refinement of component placement. This mechanism reflects the convergence behavior of the MAS and its dependency on computational budget. Figure 7.2 presents the constraint configuration tab used during experimentation.

7.2.3 Optimization Execution

Once the problem is configured, users initiate the optimization process. The MAS framework is executed in the background, and the system provides real-time feedback, including solver progress, runtime information, and intermediate results.

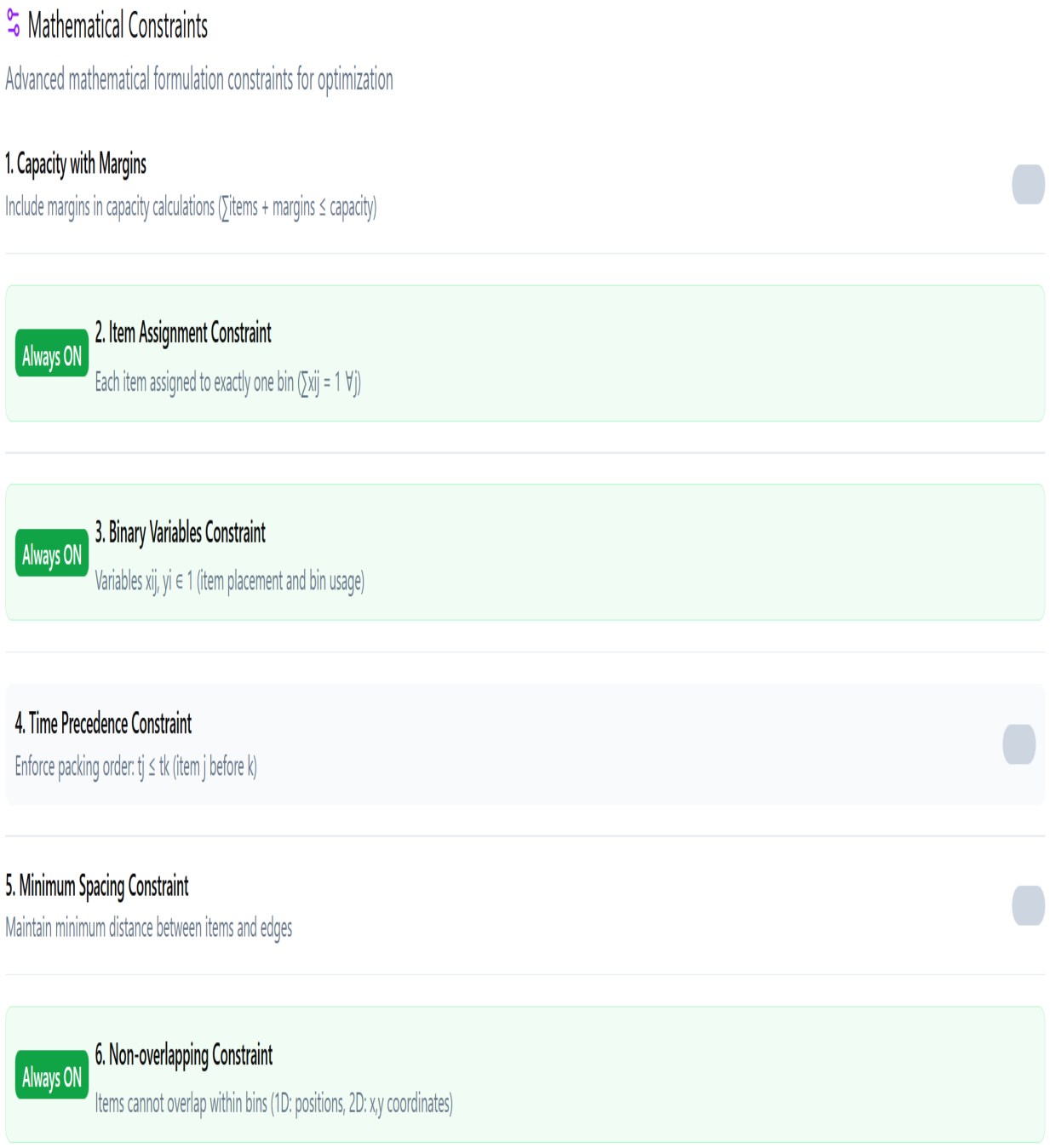

Figure 7.2: Constraint configuration interface showing geometric and operational settings.

Figure 7.3: Optimization execution interface showing solver progress and runtime feedback.

7.2.4 Result Visualization

The visualization tab presents the final nesting layout generated by the MAS optimization process. Users can inspect component placement, verify geometric compliance, and evaluate material utilization through an interactive graphical representation. Figure 7.4 shows the visualization interface displaying the final packing layout.

7.3 System Features

The prototype provides several key features that enhance usability and transparency:

- **Real-time Visualization:** Displays packing layouts and allows users to inspect item placement.

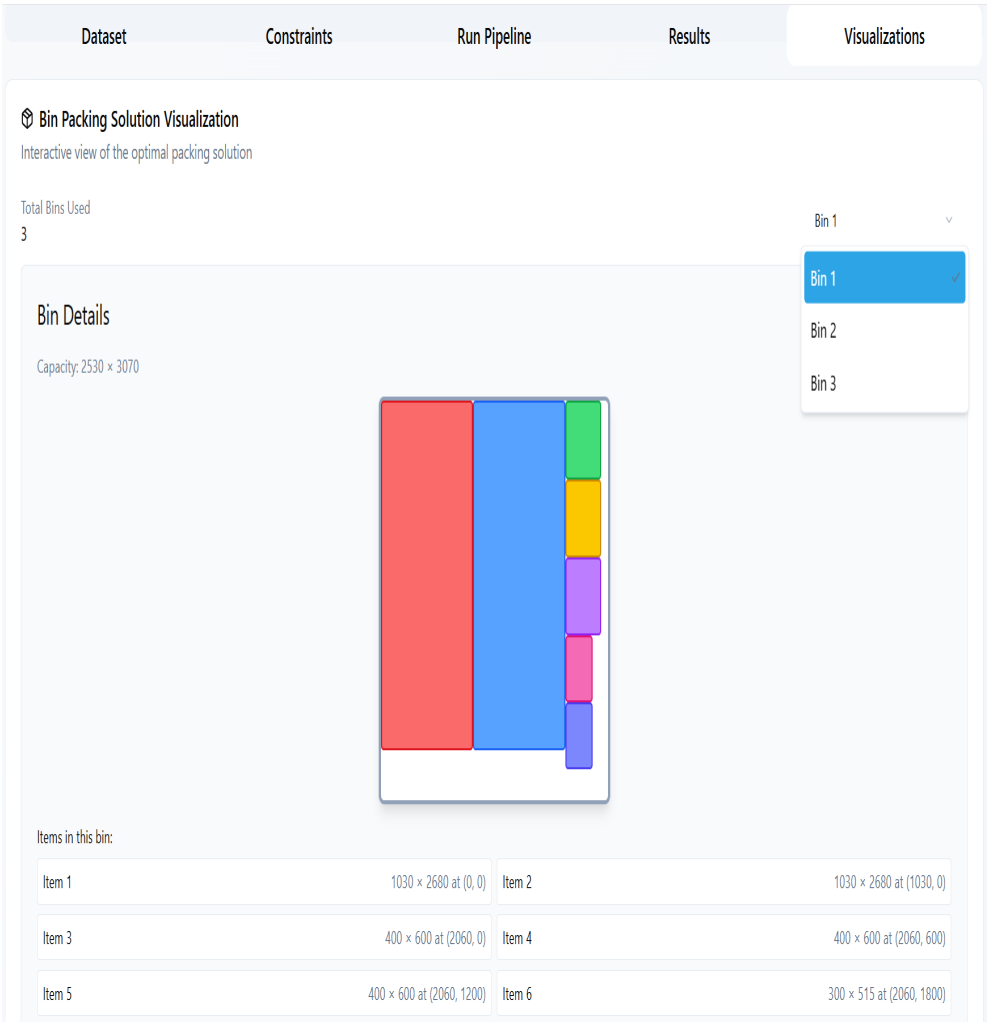

Figure 7.4: Interactive visualization of final packing configuration showing item placement and constraint satisfaction.

- **Configurable Constraints:** Enables users to define and modify problem constraints.
- **Performance Metrics:** Reports bin usage, runtime, and optimization quality.
- **Agent Transparency:** Provides insights into the decision-making process of each agent in the MAS pipeline.

These features allow users to understand both the outcome and the internal behavior of the optimization process.

7.4 Industrial Applications

The proposed MAS framework is applicable to a wide range of industrial scenarios:

7.4.1 Logistics and Transportation

The framework can be used for container loading and pallet optimization, enabling efficient utilization of space and reduction of transportation costs.

7.4.2 Manufacturing and Cutting Stock

In manufacturing environments, the system can optimize material usage in cutting and nesting problems, reducing waste and improving production efficiency.

7.4.3 E-commerce Fulfillment

The framework can be applied to cartonization and order packing, enabling efficient packaging strategies that reduce shipping costs and environmental impact.

7.5 Discussion

The prototype demonstrates that the proposed MAS framework is not only theoretically effective but also practically applicable. The integration of optimization algorithms into an interactive system provides valuable insights into the decision-making process and enables real-time application in industrial settings.

The modular design of the system allows for future extensions, such as the integration of additional optimization algorithms, support for higher-dimensional packing problems, and deployment in distributed environments.

7.6 Summary

This chapter presented the design and implementation of a web-based prototype for the proposed MAS framework. The system demonstrates the practical applicability of the approach and highlights its potential for real-world deployment across multiple industrial domains. The final chapter concludes the thesis and outlines directions for future research.

Chapter 8

Conclusion

This thesis presented an Adaptive Multi-Agent System (MAS) for solving one-dimensional (1D) and two-dimensional (2D) bin packing problems. The proposed framework integrates heuristic, metaheuristic, and exact optimization techniques within a unified and coordinated architecture. By leveraging a combination of data-driven decision-making and structured optimization strategies, the system effectively addresses the trade-off between solution quality and computational efficiency.

A key contribution of this work is the design of a five-agent architecture that decomposes the optimization process into modular and specialized components. The Reinforcement Learning (RL)-based heuristic selector enables adaptive decision-making by selecting appropriate heuristics based on instance-level features. The metaheuristic optimization agent refines solutions using Cuckoo Search, while the exact method agent provides additional

improvement through a time-bounded Branch and Bound procedure. The pipeline orchestrator ensures efficient coordination and resource allocation across all stages.

Extensive experimental evaluation on more than 3,000 benchmark instances demonstrated that the proposed MAS consistently outperforms classical heuristic and metaheuristic methods. The framework achieves up to 9% improvement in bin utilization while maintaining an optimality gap of less than 2%. Furthermore, the system achieves up to 10× faster runtime compared to exact methods, making it suitable for real-time applications.

In addition to algorithmic contributions, this thesis introduced a web-based proof-of-concept prototype that demonstrates the practical applicability of the proposed framework in industrial scenarios. The prototype enables interactive problem configuration, real-time optimization, and visualization of packing solutions, providing valuable insights into the behavior of the system.

8.1 Limitations

Despite its effectiveness, the proposed framework has several limitations. The RL-based heuristic selector relies on a relatively simple feature representation and does not fully exploit advanced deep learning techniques. The pipeline architecture is sequential, which may limit scalability and parallel execution in large-scale distributed environments. Additionally, the current framework

assumes static problem instances and does not account for dynamic or online bin packing scenarios.

8.2 Future Work

Several directions can be explored to extend this work. First, the integration of more advanced reinforcement learning models, such as deep reinforcement learning, could improve decision-making capabilities and adaptability. Second, extending the framework to handle dynamic and online bin packing problems would enhance its applicability in real-time environments. Third, the incorporation of additional constraints, such as precedence relations and weight distribution, would enable more realistic industrial modeling.

Further research may also explore extending the framework to three-dimensional (3D) bin packing problems and investigating parallel or distributed implementations of the multi-agent system. Finally, integrating additional optimization techniques and expanding the agent architecture could further improve performance and scalability.

Bibliography

[1] Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. Machine learning for combinatorial optimization. *European Journal of Operational Research*, 2021.

[2] J. Bennell and J. Oliveira. 2d bin packing with precedence constraints. *European Journal of Operational Research*, 2020.

[3] E. Bischoff. 3d packing problems. *European Journal of Operational Research*, 2006.

[4] M. Boschetti and A. Mingozzi. 2d bin packing with positioning. *European Journal of Operational Research*, 2012.

[5] Edmund et al. Burke. Hyper-heuristics: A survey. *European Journal of Operational Research*, 2013.

[6] E. G. Coffman, J. Csirik, D. S. Johnson, and G. J. Woeginger. Bin packing approximation algorithms: Survey and classification. *Handbook of Combinatorial Optimization*, 1997.

[7] E. G. Coffman, M. R. Garey, and D. S. Johnson. Approximation algorithms for bin packing: A survey. *Approximation Algorithms for NP-hard Problems*, 1984.

[8] T. et al. Crainic. 3d bin packing heuristics. *INFORMS Journal on Computing*, 2008.

[9] M. Delorme. Bpplib library, 2018.

[10] M. Delorme, M. Iori, and S. Martello. Bin packing and cutting stock problems. *European Journal of Operational Research*, 2018.

[11] DHL. Opticarton ai packaging, 2022.

[12] Marco Dorigo and Luca Gambardella. Ant colony system. *IEEE Transactions on Evolutionary Computation*, 1996.

[13] EURO Special Interest Group on Cutting and Packing. Esicup benchmark library, 2015. https://www.esicup.org.

[14] Emanuel Falkenauer. A hybrid grouping genetic algorithm for bin packing. *Journal of Heuristics*, 1996.

[15] M. et al. Gasse. Deep reinforcement learning for combinatorial optimization. *NeurIPS*, 2021.

[16] John H. Holland. Adaptation in natural and artificial systems. *University of Michigan Press*, 1975.

[17] V. Horky. Multi-agent system for bin packing. *International Journal of Computer Applications*, 2012.

[18] Nicholas R. et al. Jennings. A roadmap of agent research. *Autonomous Agents and Multi-Agent Systems*, 1998.

[19] D. S. Johnson. Approximation algorithms for combinatorial problems. *Journal of Computer and System Sciences*, 1974.

[20] E. et al. Khalil. Machine learning for combinatorial optimization. *NeurIPS*, 2020.

[21] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 1983.

[22] A. Lodi, S. Martello, and D. Vigo. Exact solution techniques for 2d packing. *European Journal of Operational Research*, 2007.

[23] Andrea Lodi, Silvano Martello, and Daniele Vigo. Heuristic algorithms for the two-dimensional bin packing problem. *European Journal of Operational Research*, 141(2):410–420, 1999.

[24] S. Martello et al. Bed-bpp dataset, 2021. Benchmark dataset for 2D bin packing problems.

[25] S. Martello and D. Vigo. Exact solution of the two-dimensional finite bin packing problem. *INFORMS Journal on Computing*, 1998.

[26] Silvano Martello and Paolo Toth. *Knapsack Problems: Algorithms and Computer Implementations*. Wiley, 1990.

[27] Volodymyr et al. Mnih. Human-level control through deep reinforcement learning. *Nature*, 2015.

[28] Eneko Osaba, Esther Villar-Rodriguez, and Sebastian V. Romero. Benchmark dataset and instance generator for real-world three-dimensional bin packing problems. *Data in Brief*, 49:109309, 2023.

[29] John R. Rice. The algorithm selection problem. *Advances in Computers*, 1976.

[30] A. Scholl, R. Klein, and C. Jürgens. Bison: A fast hybrid procedure for exactly solving the one-dimensional bin packing problem. *Computers & Operations Research*, 1997.

[31] David et al. Silver. Mastering the game of go. *Nature*, 2016.

[32] J. et al. Song. Machine learning for multi-dimensional bin packing. *IEEE Access*, 2022.

[33] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2018.

[34] El-Ghazali Talbi. *Metaheuristics: From Design to Implementation*. Wiley, 2009.

[35] thyssenkrupp. Industrial dataset, 2025.

[36] G. Wäscher, H. Haußner, and H. Schumann. An improved typology of cutting and packing problems. *European Journal of Operational Research*, 2010.

[37] Gerhard Wäscher, Heike Haußner, and Holger Schumann. An improved typology of cutting and packing problems. *European Journal of Operational Research*, 183(3):1109–1130, 2007.

[38] Christopher Watkins and Peter Dayan. Q-learning. *Machine Learning*, 1992.

[39] Michael Wooldridge. *An Introduction to MultiAgent Systems*. Wiley, 2009.

[40] Xin-She Yang and Suash Deb. Cuckoo search via lévy flights. In *NaBIC*, 2009.